

Технически университет – София

Факултет електронна техника и технологии

Дисциплина: Микропроцесорна системотехника

КУРСОВ ПРОЕКТ

Тема: Управление на посоката на въртене на постояннотоков мотор чрез бутони и прекъсване

Град: София

Дата: 03.04.2026 година

Изготвил: Преслав Димитров

Факултетен номер: 941223057

Ръководител: маг. инж. Николай Кадиев

Технически Университет – София
Факултет Електронна техника и технологии

Задание за курсов проект
по
Микропроцесорна схемотехника

на студента: Преслав Димитров
фак. № 941223057 група: 69Б

Тема: Управление на посоката на въртене на постояннотоков мотор чрез бутони и прекъсване

Изходни данни:
1. Микроконтролер – избран на база указанията получени със заданието
2. Използване на GPIO и PWM
3. Използване на 5V постояннотоков мотор
4. Използване на 3 бутона – бутон старт/стоп, бутон на ляво, бутон на дясно
5. Използване на прекъсване

Съдържание:
1. Заглавна страница и задание
2. Съдържание по страници
3. Литературно проучване
4. Блокова схема с пояснение към нея
5. Проектиране на принципна схема и изчисления
5. Алгоритъм на управляващата програма (и/или сорс код)
6. Използвана литература

Указания:
Курсовият проект трябва да е с обем 10 ÷ 15 стр., шрифт 14. Фигурите и принципните схеми трябва да са лично направени. Включването на каталожни данни е нежелателно, освен когато се обяснява действието на елементите. Проектирането на вградената система включва и захранващия ѝ блок. Конструктивното проектиране на трансформатори не е задължително. Указания за избор на микроконтролер са дадени в „условия за избор на контролер и помощна литература.pdf“ изпратен заедно със заданието.
При завършване на проекта студентът трябва да го изпрати като PDF документ на nkadiev@tu-sofia.bg

Дата на задаване: 23.02.2026
Краен срок: 20.05.2026

Ръководител: .....
/маг. инж. Николай Кадиев/

1.Литературно проучване: 3
2.Избор на компоненти 5
2.1 Микроконтролер: 5
2.1.1 Логическо разпределение на функциите: 5
2.1.2 Избор на пинове: 5
2.2 Моторът: 6
2.3 H-мост: 6
2.3.1 Основна архитектура: 6
2.3.2 Избор на H-мост: 6
2.4 Избор на вид захранване: 7
3.Блокова схема: 7
3.1 Обяснение на блоковете и връзките: 8
4.Изработване на принципна схема: 8
4.1 Обяснение на принципната схема: 9
5.Сорс код: 9
5.1 Изработване на блок-алгоритъм на кода 25
5.2 Описание на програмния код 25
6.Заключение 27
6.Литература 27

Изходни данни:

1. Микроконтролер – избран на база указанията получени със заданието

2. Използване на GPIO и PWM

3. Използване на 5V постояннотоков мотор

4. Използване на 3 бутона – бутон старт/стоп, бутон на ляво, бутон на дясно

5. Използване на прекъсване

1.Литературно проучване:

Целта на настоящия проект е разработването на система за управление на посоката на въртене на постояннотоков мотор чрез използване на бутони и прекъсвания в микроконтролерна система. Това изисква и работа с GPIO и

PWM интерфейси, писане на алгоритъм на управляващата програма и проектиране на принципна схема сс модули с различно напрежение.

Устройството позволява управление на посоката на въртене на електродвигател в зависимост от подадена команда от потребителя. Тези команди се подават чрез 3 бутона: start/stop бутон, бутон на ляво и бутон на дясно.

В допълнение подобно устройство може да намери приложение в много техника от ежедневието, като се започне от кухненска техника, електрически щори и гаражни врати, детски играчки, малки роботи и вентилатори, както и в индустрията за транспортни ленти, CNC машини, роботи и други.

На пазара вече има подобна технология, като например: Arduino motor shield, Industrial motor drivers и PLC управление.

Моят проект ще бъде реализиран с използването на:микроконтролер, електрически мототр, 3 бутона и H-мост за оправлението на електрическото моторче.

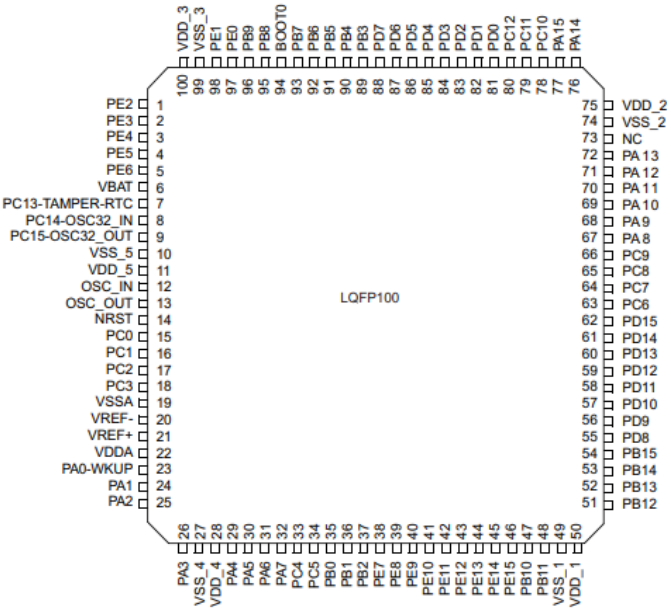
2.Избор на компоненти

2.1 Микроконтролер:

Избираме микроконтролер STM32F103C8T6.

Причини:

- 3.3V логика
- хардуерен PWM
- Външни прекъсвания на почти всеки GPIO



2.1.1 Логическо разпределение на функциите:

Имаме нужда от:

- 3 входа за бутоните
- 3 изхода за H-мост(2 за посока, 1 PMW за скорост)
- Захранване
- Маса

2.1.2 Избор на пинове:

- PWM изход: PA8 – TIM1_CH1

Обяснение: хардуерен PWM; Advanced timer, подходящ за управление на мощности.

- Управление на посока: PB0 – GPIO Output; PB1 – GPIO Output

Обяснение: двата пина ще определят въртенето в двете посоки.

- Бутон Start/Stop с прекъсване: PA0 – EXTI0

Обяснение: При натискане ще се генерира прекъсване.

- Бутони Left/Right: PA1 – GPIO Input; PA2 – GPIO Input

Обяснение: Бутони с които се управлява въртенето в двете посоки

- Захранване:

VDD = 3.3V;

VSS = GND;

NRST - reset бутон;

BOOT0 - към GND

2.2 Моторът:

- 5V DC мотор;
- Номинален ток - 300-500mA;
- Пусков ток - до 1A;

2.3 H-мост:

2.3.1 Основна архитектура:

- 4 силови транзистора(MOSFET);
- 2 управляващи сигнала от микроконтролера;
- 1 PWM сигнал ;

2.3.2 Избор на H-мост:

L293D - Моторен драйвер съдържащ 2 H-моста от които ще ползваме един.

Защо е подходящ:

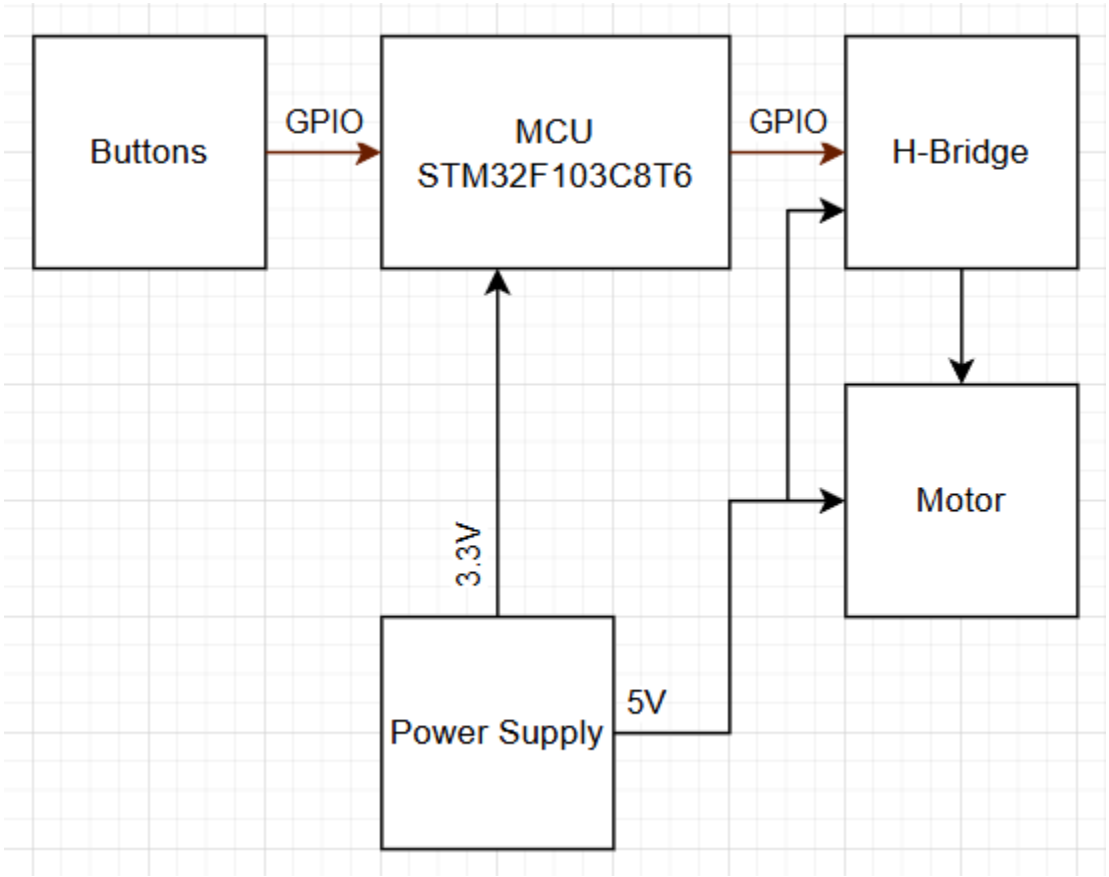
- лесно управление от MCU;
- има вградени защити;
- надежден;

- поддържа моторно напрежение от 4.5V-36V(на нас ни трябват 5V)

2.4 Избор на вид захранване:

За целите на нашата задача ни трябва източник на напрежение 5V за драйвера на електрическия мотор и за самия електрически мотор, а за микроконтролера и периферията му са нужни 3.3V. Логично решение е да се направи захранващо USB със стабилизатор за напрежение. Понеже USB може да отдава 5V това го прави перфектно за захранване на мотора и драйва му. За захранване на микроконтролера и периферията му ще ни е нужен линеен стабилизатор на напрежение който да превръща 5V от USB в 3.3V. Такъв линеен стабилизатор е AMS1117-3.3.

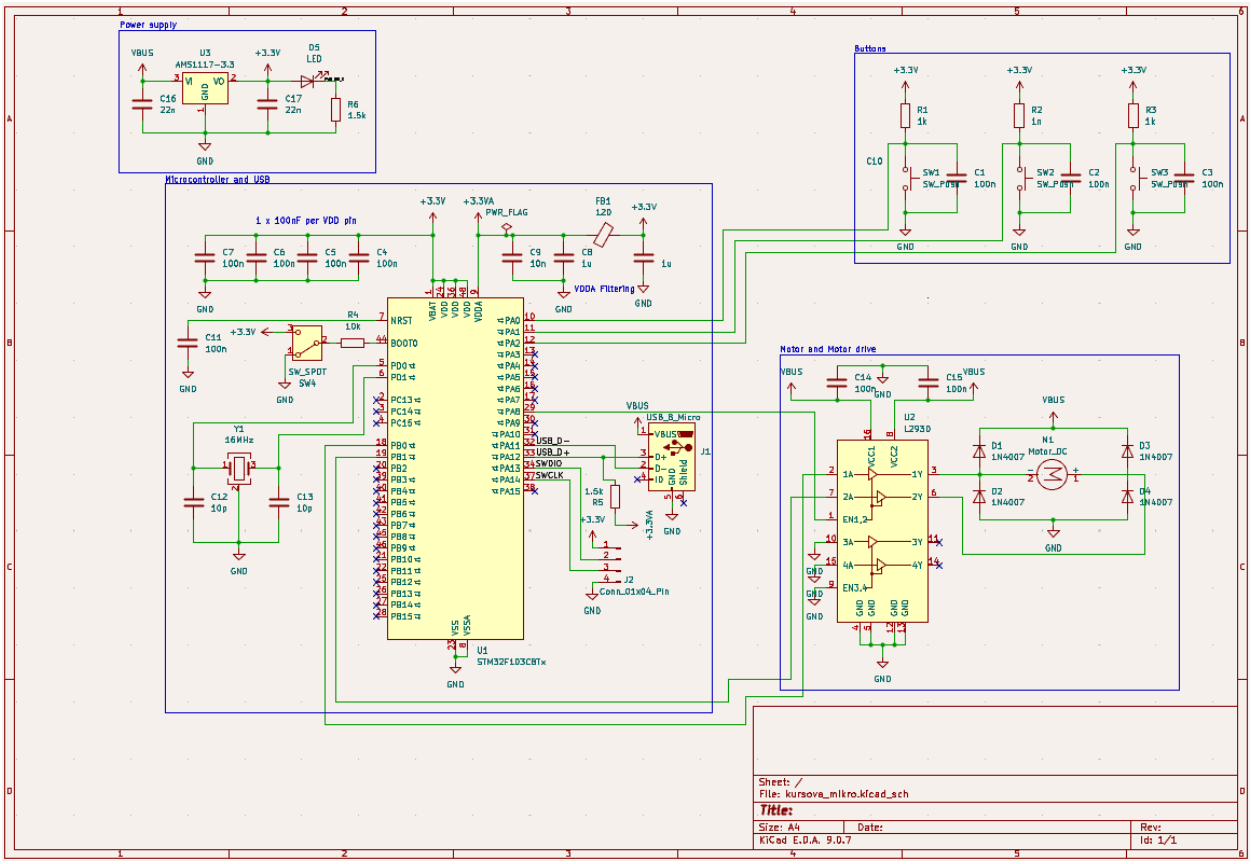
3.Блокова схема:



3.1 Обяснение на блоковете и връзките:

- MCU STM32F103C8T6 - микроконтролер;
- Power supply - захранване;
- Button right - бутон за смяна на посоката на въртене на мотора свързан в GPIO пин в MCU;
- Button left - бутон за смяна на посоката на въртене на мотора свързан в GPIO пин в MCU;
- Button Start/Stop - бутон, който пуска и спира постояннотоковия мотор свързан в GPIO пин в MCU;
- H-Bridge - електронна схема, която позволява да се управлява посоката на въртене на DC мотора;
- Motor - постояннотоков мотор;

4.Изработване на принципна схема:



Към имейла с курсовия проект е приложен pdf файл на принципната схема за по-лесно преглеждане.

4.1 Обяснение на принципната схема:

USB_B_Micro е USB което се използва директно за захранване на мотора и драйва и се преобразува в 3.3V за захранване на остатъка на схемата. J2 извод от контролера който позволява тестване или захранване на външна периферия на 3.3V. AMS1117-3.3 е линейният стабилизатор който е отговорен за преобразуване на напрежението от 5V в 3.3V. Към него има кондензатори C16 и C17 които са отговорни за филтриране на входното и изходното напрежение на схемата. Кондензатори от C4 до C7 филтрират входното напрежение за основната част на микроконтролера, сложени по препоръка на производителя(по един кондензатор за пин). Пин VDDA е пинът от който се захранва аналоговата част на микроконтролера. При захранването му е използван FB1(Ferrite Beat) за чистене на високочестотния шум, C8 и C9 са допълнителна филтрация за да се осигури възможно най-чист сигнал. STM32F103C8T6 има вграден тактов генератор, но той не е надежден. Поради тази причина използване кварца Y1 с кондензатори C12 и C13 за изграждане на тактовия сигнал на микроконтролера. Групата SW_SPDT и R4 се използва за стартиране на микроконтролера. C11 се използва за рестартиране на микроконтролера. U2 е моторният драйвер, като се използва едната половина от него. C14 и C15 са кондензатори които филтрират захранващото напрежение за моторния драйвер. M1 е моторът а D1 до D4 са защитни диоди.

5.Сорс код:

```
/* USER CODE BEGIN Header */
/**
********************************************************************************
  * @file           : main.c
  * @brief          : Main program body
```

```
******************************************************************************
  * @attention
  *
  * <h2><center>&copy; Copyright (c) 2026 STMicroelectronics.
  * All rights reserved.</center></h2>
  *
  * This software component is licensed by ST under BSD 3-Clause license,
  * the "License"; You may not use this file except in compliance with the
  * License. You may obtain a copy of the License at:
  *                        opensource.org/licenses/BSD-3-Clause
  *

******************************************************************************
  */
/* USER CODE END Header */
/* Includes ------------------------------------------------------------------*/
#include "main.h"

/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */

/* USER CODE END Includes */
```

```
/* Private typedef -----------------------------------------------------------*/
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define ------------------------------------------------------------*/
/* USER CODE BEGIN PD */
/* USER CODE END PD */

/* Private macro -------------------------------------------------------------*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables ---------------------------------------------------------*/
TIM_HandleTypeDef htim1;

PCD_HandleTypeDef hpcd_USB_FS;

/* USER CODE BEGIN PV */
volatile uint8_t motor_enabled = 0;
volatile uint8_t direction = 0;
/* USER CODE END PV */
```

```
/* Private function prototypes -----------------------------------------------*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_USB_PCD_Init(void);
static void MX_TIM1_Init(void);
/* USER CODE BEGIN PFP */

/* USER CODE END PFP */

/* Private user code ---------------------------------------------------------*/
/* USER CODE BEGIN 0 */

/* USER CODE END 0 */

/**
  * @brief  The application entry point.
  * @retval int
  */
int main(void)
{
  /* USER CODE BEGIN 1 */

  /* USER CODE END 1 */
```

```
/* MCU Configuration--------------------------------------------------------*/

/* Reset of all peripherals, Initializes the Flash interface and the Systick. */
HAL_Init();

/* USER CODE BEGIN Init */

/* USER CODE END Init */

/* Configure the system clock */
SystemClock_Config();

/* USER CODE BEGIN SysInit */

/* USER CODE END SysInit */

/* Initialize all configured peripherals */
MX_GPIO_Init();
MX_USB_PCD_Init();
MX_TIM1_Init();
/* USER CODE BEGIN 2 */
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);
/* USER CODE END 2 */
```

```
  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
        if(motor_enabled)
            {
              // PWM ~70% скорост
              __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, 700);

              if(direction == 1) // LEFT
              {
                 HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_SET);
                                    HAL_GPIO_WritePin(GPIOB,   GPIO_PIN_1,
GPIO_PIN_RESET);
              }
              else if(direction == 2) // RIGHT
              {
                                    HAL_GPIO_WritePin(GPIOB,   GPIO_PIN_0,
GPIO_PIN_RESET);
                 HAL_GPIO_WritePin(GPIOB, GPIO_PIN_1, GPIO_PIN_SET);
              }
            }
            else
            {
              // Stop motor
```

```
            __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, 0);
            HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_RESET);
            HAL_GPIO_WritePin(GPIOB, GPIO_PIN_1, GPIO_PIN_RESET);
          }
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
  }
  /* USER CODE END 3 */
}

/**
  * @brief System Clock Configuration
  * @retval None
  */
void SystemClock_Config(void)
{
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
  RCC_PeriphCLKInitTypeDef PeriphClkInit = {0};

  /** Initializes the RCC Oscillators according to the specified parameters
  * in the RCC_OscInitTypeDef structure.
  */
```

```
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
  RCC_OscInitStruct.HSEState = RCC_HSE_ON;
  RCC_OscInitStruct.HSEPredivValue = RCC_HSE_PREDIV_DIV2;
  RCC_OscInitStruct.HSIState = RCC_HSI_ON;
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
  RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
  RCC_OscInitStruct.PLL.PLLMUL = RCC_PLL_MUL9;
  if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
  {
    Error_Handler();
  }
  /** Initializes the CPU, AHB and APB buses clocks
  */
                                  RCC_ClkInitStruct.ClockType                  =
RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                        |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

    if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) !=
HAL_OK)
  {
    Error_Handler();
```

```
  }
  PeriphClkInit.PeriphClockSelection = RCC_PERIPHCLK_USB;
  PeriphClkInit.UsbClockSelection = RCC_USBCLKSOURCE_PLL_DIV1_5;
  if (HAL_RCCEx_PeriphCLKConfig(&PeriphClkInit) != HAL_OK)
  {
    Error_Handler();
  }
}

/**
  * @brief TIM1 Initialization Function
  * @param None
  * @retval None
  */
static void MX_TIM1_Init(void)
{

  /* USER CODE BEGIN TIM1_Init 0 */

  /* USER CODE END TIM1_Init 0 */

  TIM_MasterConfigTypeDef sMasterConfig = {0};
  TIM_OC_InitTypeDef sConfigOC = {0};
  TIM_BreakDeadTimeConfigTypeDef sBreakDeadTimeConfig = {0};
```

```
  /* USER CODE BEGIN TIM1_Init 1 */

  /* USER CODE END TIM1_Init 1 */
  htim1.Instance = TIM1;
  htim1.Init.Prescaler = 72-1;
  htim1.Init.CounterMode = TIM_COUNTERMODE_UP;
  htim1.Init.Period = 1000-1;
  htim1.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
  htim1.Init.RepetitionCounter = 0;
  htim1.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
  if (HAL_TIM_PWM_Init(&htim1) != HAL_OK)
  {
    Error_Handler();
  }
  sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
  sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
    if (HAL_TIMEx_MasterConfigSynchronization(&htim1, &sMasterConfig) !=
HAL_OK)
  {
    Error_Handler();
  }
  sConfigOC.OCMode = TIM_OCMODE_PWM1;
  sConfigOC.Pulse = 0;
  sConfigOC.OCPolarity = TIM_OCPOLARITY_HIGH;
```

```
  sConfigOC.OCNPolarity = TIM_OCNPOLARITY_HIGH;
  sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;
  sConfigOC.OCIdleState = TIM_OCIDLESTATE_RESET;
  sConfigOC.OCNIdleState = TIM_OCNIDLESTATE_RESET;
  if (HAL_TIM_PWM_ConfigChannel(&htim1, &sConfigOC, TIM_CHANNEL_1) != HAL_OK)
  {
    Error_Handler();
  }
  sBreakDeadTimeConfig.OffStateRunMode = TIM_OSSR_DISABLE;
  sBreakDeadTimeConfig.OffStateIDLEMode = TIM_OSSI_DISABLE;
  sBreakDeadTimeConfig.LockLevel = TIM_LOCKLEVEL_OFF;
  sBreakDeadTimeConfig.DeadTime = 0;
  sBreakDeadTimeConfig.BreakState = TIM_BREAK_DISABLE;
  sBreakDeadTimeConfig.BreakPolarity = TIM_BREAKPOLARITY_HIGH;
  sBreakDeadTimeConfig.AutomaticOutput = TIM_AUTOMATICOUTPUT_DISABLE;
  if (HAL_TIMEx_ConfigBreakDeadTime(&htim1, &sBreakDeadTimeConfig) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN TIM1_Init 2 */

  /* USER CODE END TIM1_Init 2 */
```

```
  HAL_TIM_MspPostInit(&htim1);

}

/**
  * @brief USB Initialization Function
  * @param None
  * @retval None
  */
static void MX_USB_PCD_Init(void)
{

  /* USER CODE BEGIN USB_Init 0 */

  /* USER CODE END USB_Init 0 */

  /* USER CODE BEGIN USB_Init 1 */

  /* USER CODE END USB_Init 1 */
  hpcd_USB_FS.Instance = USB;
  hpcd_USB_FS.Init.dev_endpoints = 8;
  hpcd_USB_FS.Init.speed = PCD_SPEED_FULL;
  hpcd_USB_FS.Init.low_power_enable = DISABLE;
  hpcd_USB_FS.Init.lpm_enable = DISABLE;
```

```
  hpcd_USB_FS.Init.battery_charging_enable = DISABLE;
  if (HAL_PCD_Init(&hpcd_USB_FS) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN USB_Init 2 */

  /* USER CODE END USB_Init 2 */

}

/**
  * @brief GPIO Initialization Function
  * @param None
  * @retval None
  */
static void MX_GPIO_Init(void)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};

  /* GPIO Ports Clock Enable */
  __HAL_RCC_GPIOD_CLK_ENABLE();
  __HAL_RCC_GPIOA_CLK_ENABLE();
  __HAL_RCC_GPIOB_CLK_ENABLE();
```

```
  /*Configure GPIO pin Output Level */
                HAL_GPIO_WritePin(GPIOB,        Left_output_Pin|Right_output_Pin,
GPIO_PIN_RESET);

  /*Configure GPIO pins : PA0 PA1 PA2 */
  GPIO_InitStruct.Pin = GPIO_PIN_0|GPIO_PIN_1|GPIO_PIN_2;
  GPIO_InitStruct.Mode = GPIO_MODE_IT_FALLING;
  GPIO_InitStruct.Pull = GPIO_PULLUP;
  HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

  /*Configure GPIO pins : Left_output_Pin Right_output_Pin */
  GPIO_InitStruct.Pin = Left_output_Pin|Right_output_Pin;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

}

/* USER CODE BEGIN 4 */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
        HAL_Delay(50); // debounce
    if(GPIO_Pin == GPIO_PIN_0) // Start/Stop
```

```
    {
        motor_enabled = !motor_enabled;
    }
    if(GPIO_Pin == GPIO_PIN_1) // Left
    {
        direction = 1;
    }
    if(GPIO_Pin == GPIO_PIN_2) // Right
    {
        direction = 2;
    }
}
/* USER CODE END 4 */

/**
  * @brief  This function is executed in case of error occurrence.
  * @retval None
  */
void Error_Handler(void)
{
  /* USER CODE BEGIN Error_Handler_Debug */
  /* User can add his own implementation to report the HAL error return state */

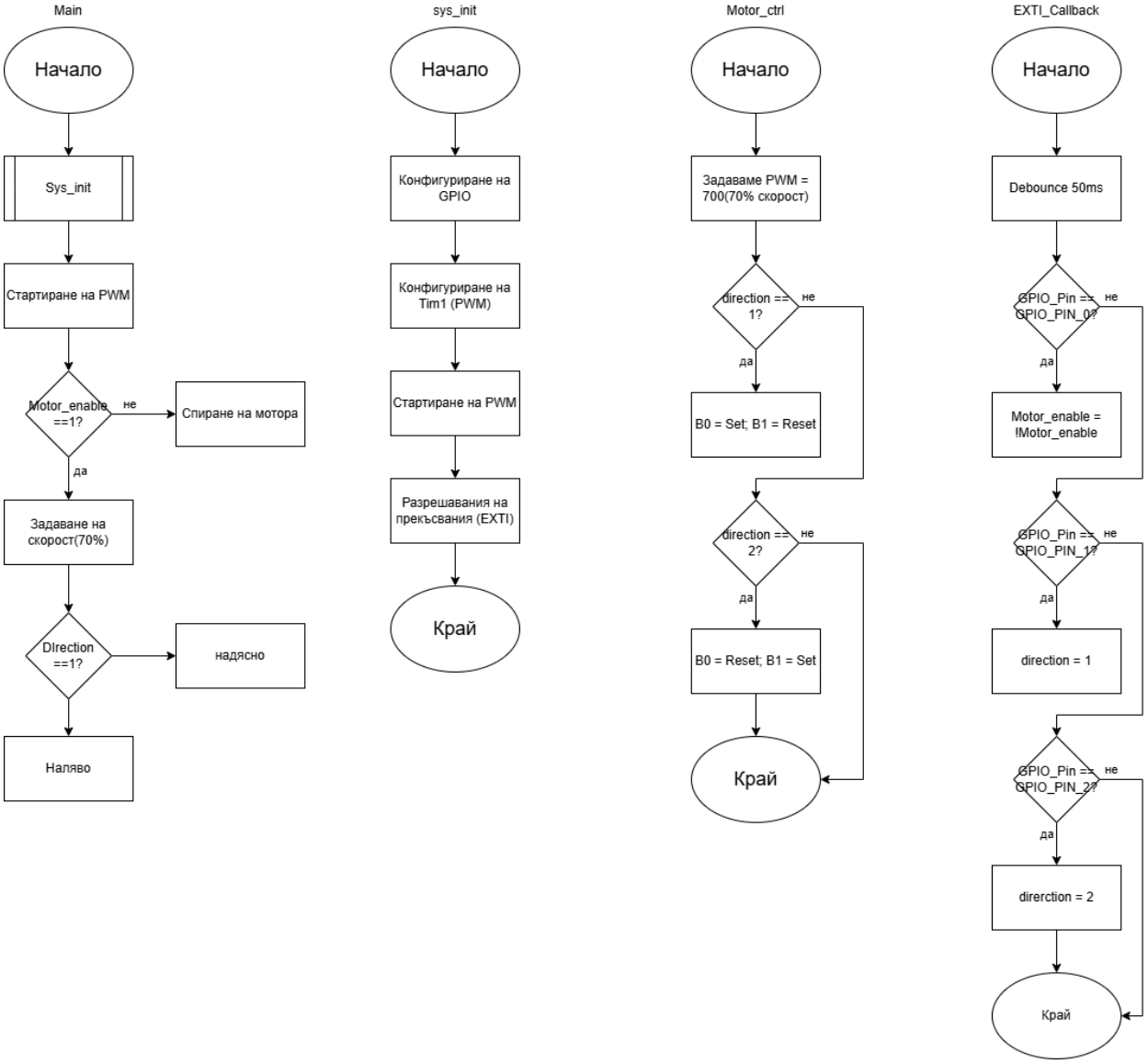
  /* USER CODE END Error_Handler_Debug */
```

```
}

#ifdef  USE_FULL_ASSERT
/**
  * @brief  Reports the name of the source file and the source line number
  *         where the assert_param error has occurred.
  * @param  file: pointer to the source file name
  * @param  line: assert_param error line source number
  * @retval None
  */
void assert_failed(uint8_t *file, uint32_t line)
{
  /* USER CODE BEGIN 6 */
  /* User can add his own implementation to report the file name and line number,
     tex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
  /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */

/************************ (C) COPYRIGHT STMicroelectronics *****END OF FILE****/
```

5.1 Изработване на блок-алгоритъм на кода



5.2 Описание на програмния код

Програмата реализира управление на електродвигател чрез микроконтролер от серията STM32. Управлението се осъществява чрез използване на широчинно-импулсна модулация (PWM) за регулиране на скоростта и цифрови изходи за задаване на посоката на въртене. Входната информация се подава чрез бутони, свързани към GPIO пинове, които работят в режим на прекъсване.

След стартиране на програмата се извършва инициализация на всички необходими периферни устройства – GPIO портове, таймер (TIM1) и USB модул. Таймерът е конфигуриран в PWM режим, като чрез него се генерира сигнал за управление на скоростта на двигателя.

Основната логика на програмата се изпълнява в безкраен цикъл. В него се проверява състоянието на променливата motor_enabled, която определя дали двигателят трябва да работи. Ако моторът е активиран, се задава PWM сигнал с коефициент на запълване около 70%, което определя скоростта на въртене. В същото време, в зависимост от стойността на променливата direction, се задава логическо състояние на два GPIO пина, чрез които се определя посоката на въртене на двигателя (наляво или надясно).

Ако моторът не е активиран, PWM сигналът се занулява, което води до спиране на двигателя, а изходните пинове се нулират, осигурявайки безопасно състояние.

HAL_TIM_PWM_Start() - Функцията стартира генерирането на PWM сигнал от зададения таймер и канал.

__HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, value); - Задава стойността на регистъра за сравнение (CCR), което определя duty cycle на PWM сигнала.

HAL_GPIO_WritePin(GPIOB, GPIO_PIN_X, GPIO_PIN_SET/RESET); - Задава логическо състояние на даден GPIO пин. SET -> логическо 1(HIGH)

RESET -> логическа 0(LOW)

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) - Callback функция, която се извиква автоматично при външно прекъсване от GPIO пин. Разпознава кой бутон е натиснат и изпълнява съответната логика.

HAL_Delay(50); - Спира изпълнението на програмата за определено време. Използва се за debounce на бутоните.

6.Заключение

Целта на поставения курсов проект беше проектирането и реализирането на система за управление на посоката на електрически мотор с помощта на микроконтролер, бутони и PWM сигнал с прекъсване.

В рамките на проекта бе разработена блокова схема, избрани отделните части на системата и построена пълна принципна схема. Разработеният сос код осигурява надеждно управление на на мотора, като interrupts позволява бърза реакция на входните сигнали.

В заключение, разработената система изпълнява поставените цели и показва ефективно управление на постояннотоковото моторче.

6.Литература

https://www.st.com/resource/en/datasheet/stm32f103c8.pdf

https://www.st.com/en/development-tools/stm32cubeide.html#overview

https://www.youtube.com/@stm32world